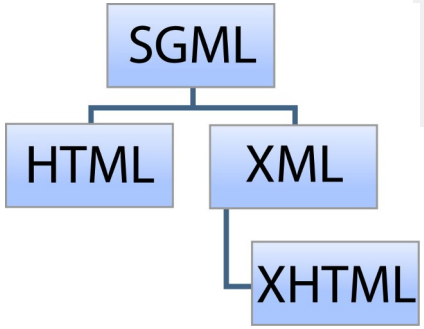
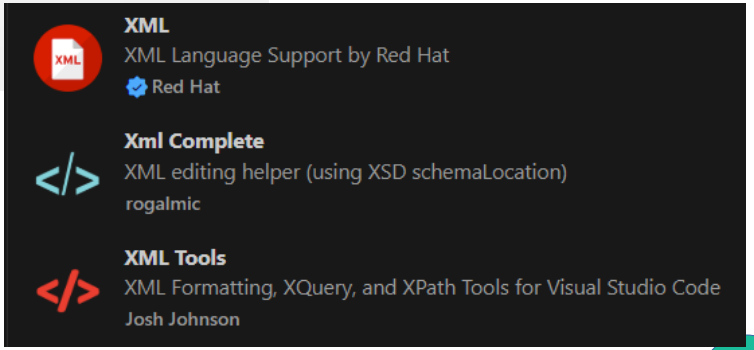
UT4 DTD y XML Schema. Documentos válidos
Curso 2024/2025 LENGUAJE DE MARCAS 1º DAW





UT4 DTD y XML Schema. Documentos válidos

ÍNDICE

1. DTD (Document Type Definition)
    1.1. ¿Qué es un DTD?
    1.2. Declaración de DTD
        1.2.1 Declaración de Elementos en una DTD
        1.2.2 Declaración de Atributos en una DTD
        1.2.3 Declaración de Entidades en una DTD
2. Como validar documento XML asociado a un DTD

ÍNDICE

3. XSD (XML Schema Definition)
- 3.1. ¿Qué es un XSD?
- 3.2. Diferencias entre DTD y XSD
- 3.3. Creación de un XSD
  - 3.3.1. Estructura de un *schema*
  - 3.3.2. Cómo vincular un esquema XSD a un documento XML
- 3.4. Definición de un elemento
- 3.5. Tipos de elementos
  - 3.5.1. Elementos simples <xs:element>
  - 3.5.2. Elementos complejos <xs:complexType>
  - 3.5.3. Elementos simples personalizados <xs:simpleType>
- 3.6. Validación de un documento XML con XSD

ÍNDICE

4. XPATH (XML Path Language)
- 4.1. ¿Qué es XPath?
- 4.2. Usos de XPath
- 4.3. ¿Cómo ejecuto una consulta XPath?
- 4.4. El árbol XPath
- 4.5. Tipos de nodos es un árbol XPath
  - 4.5.1. Elementos simples <xs:element>
  - 4.5.2. Elementos complejos <xs:complexType>
  - 4.5.3. Elementos simples personalizados <xs:simpleType>
- 4.6. Sintaxis y notación XPath
- 4.7. Trayecto o ruta de búsqueda
  - 4.7.1. Ejes
  - 4.7.2. Nodos de Comprobación o de búsqueda
  - 4.7.3. Predicados
  - 4.7.4. Operadores aritméticos, lógicos y de comparación.

1.1. ¿Qué es un DTD?

Un DTD (Document Type Definition) es un conjunto de reglas que define la estructura y validación de un documento XML.

Permite especificar:

- ✔ Qué elementos y atributos pueden existir en el XML.
- ✔ El orden y jerarquía de los elementos.
- ✔ Cuántas veces puede aparecer un elemento.
- ✔ Qué tipo de datos pueden contener los elementos y atributos.

💡 ¿Para qué sirve un DTD?

- ✔ Garantiza que un archivo XML siga una estructura definida.
- ✔ Facilita la interoperabilidad entre sistemas al mantener un formato estándar.
- ✔ Permite validar XML para asegurarse de que cumple las reglas establecidas.

1.2. Declaración de DTD

Una DTD es como un CSS, puede ir en el mismo archivo XML (DTD interno) o puede ir en uno separado (DTD externo). Para poder subirlos al validador, meteremos la DTD junto con el XML.

1.- DTD Interno:

Se coloca dentro de la declaración <!DOCTYPE> en el mismo documento XML

```
<?xml version="1.0" encoding="UTF-8"?>
<!DOCTYPE libreria [
    <!ELEMENT libreria (libro+)>
    <!ELEMENT libro (titulo, autor, anio, precio)>
    <!ELEMENT titulo (#PCDATA)>
    <!ELEMENT autor (#PCDATA)>
    <!ELEMENT anio (#PCDATA)>
    <!ELEMENT precio (#PCDATA)>
]>
<libreria>
    <libro>
        <titulo>El Principito</titulo>
        <autor>Antoine de Saint-Exupéry</autor>
        <anio>1943</anio>
        <precio>15.99</precio>
    </libro>
</libreria>
```

2.- DTD Externo:

Se guarda en un archivo separado (libreria.dtd) y se enlaza desde el XML.

Archivo libreria.dtd:

```
<!ELEMENT libreria (libro+)>
<!ELEMENT libro (titulo, autor, anio, precio)>
<!ELEMENT titulo (#PCDATA)>
<!ELEMENT autor (#PCDATA)>
<!ELEMENT anio (#PCDATA)>
<!ELEMENT precio (#PCDATA)>
```

Archivo libreria.xml:

```
<?xml version="1.0" encoding="UTF-8"?>
<!DOCTYPE libreria SYSTEM "libreria.dtd">
<libreria>
    <libro>
        <titulo>El Principito</titulo>
        <autor>Antoine de Saint-Exupéry</autor>
        <anio>1943</anio>
        <precio>15.99</precio>
    </libro>
</libreria>
```

1.2.1 Declaración de Elementos en una DTD

Para declarar un elemento en una DTD se utiliza la siguiente sintaxis:

```
<!ELEMENT nombre_del_elemento tipo_de_contenido>
```

En el tipo de contenido se especifica el contenido permitido en el elemento, pudiendo ser:

1) Texto, **(#PCDATA)**.
2) Otros elementos (hijos).
3) Estar vacío, **EMPTY**.
4) Mixto (texto y otros elementos), **ANY**

1.2.1 Declaración de Elementos en una DTD

1) El contenido de un elemento puede ser texto - **(#PCDATA)**

EJEMPLO: En el siguiente documento XML, el elemento "ciudad" puede contener cualquier texto (cadena de caracteres) **(#PCDATA)**:

```
<?xml version="1.0" encoding="UTF-8"?>
<!DOCTYPE ciudad [
   <!ELEMENT ciudad (#PCDATA)>
]>

<ciudad>Roma</ciudad>
```

Escribiendo **#PCDATA** (Parsed Character Data) entre paréntesis "()", se ha indicado que el elemento "ciudad" puede contener una cadena de caracteres analizable.

1.2.1 Declaración de Elementos en una DTD

2) Un elemento puede contener a **otros elementos hijos**

EJEMPLO: En el siguiente ejemplo, el elemento "ciudad" contiene a los elementos hijos "nombre" y "pais":

```
<?xml version="1.0" encoding="UTF-8"?>
<!DOCTYPE ciudad [
   <!ELEMENT ciudad (nombre, pais)>
   <!ELEMENT nombre (#PCDATA)>
   <!ELEMENT pais (#PCDATA)>
]>

<ciudad>
  <nombre>Roma</nombre>
  <pais>Italia</pais>
</ciudad>
```

1.2.1 Declaración de Elementos en una DTD

3) Un elemento puede no contener contenido (estar vacío) - **EMPTY**

EJEMPLO: En la DTD interna del siguiente documento XML, se ha declarado el elemento "mayor_de_edad" como vacío, **EMPTY**. Por tanto, debe escribirse <mayor_de_edad/>:

```
<?xml version="1.0" encoding="UTF-8"?>
<!DOCTYPE persona [
  <!ELEMENT persona (nombre, mayor_de_edad, ciudad)>
  <!ELEMENT nombre (#PCDATA)>
  <!ELEMENT mayor_de_edad EMPTY>
  <!ELEMENT ciudad (#PCDATA)>
]>

<persona>
  <nombre>Elsa</nombre>
  <mayor_de_edad/>
  <ciudad>Pamplona</ciudad>
</persona>
```

Nota: los elementos vacíos no pueden tener contenido, pero sí pueden tener atributos.

1.2.1 Declaración de Elementos en una DTD

4) Un elemento puede definirse para contener contenido mixto - **ANY**

EJEMPLO: En la DTD interna de los siguientes documentos XML, se ha indicado que el elemento "persona" puede contener texto y otros elementos, es decir, contenido mixto, **ANY** y con esto se permite multiples tipos de documentos XML diferentes que pueden ser válidos.

```
<?xml version="1.0" encoding="UTF-8"?>
<!DOCTYPE persona [
  <!ELEMENT persona ANY>
  <!ELEMENT nombre (#PCDATA)>
  <!ELEMENT ciudad (#PCDATA)>
]>

<persona>
  <nombre>Elsa</nombre> vive en <ciudad>Pamplona</ciudad>.
</persona>
```

```
<?xml version="1.0" encoding="UTF-8"?>
<!DOCTYPE persona [
  <!ELEMENT persona ANY>
  <!ELEMENT nombre (#PCDATA)>
  <!ELEMENT ciudad (#PCDATA)>
]>

<persona>
  <nombre>Elsa</nombre> vive en Pamplona.
</persona>
```

```
<?xml version="1.0" encoding="UTF-8"?>
<!DOCTYPE persona [
  <!ELEMENT persona ANY>
  <!ELEMENT nombre (#PCDATA)>
  <!ELEMENT ciudad (#PCDATA)>
]>

<persona>
  <nombre>Elsa</nombre>
</persona>
```

Incluso, si el elemento "persona" estuviese vacío, como en este último ejemplo el documento también sería válido.

```
<?xml version="1.0" encoding="UTF-8"?>
<!DOCTYPE persona [
  <!ELEMENT persona ANY>
  <!ELEMENT nombre (#PCDATA)>
  <!ELEMENT ciudad (#PCDATA)>
]>

<persona/>
```

1.2.1 Declaración de Elementos en una DTD

4) Un elemento puede definirse para contener contenido mixto - **ANY**

Obsérvese que, por ejemplo, también sería válido el siguiente documento XML:

```
<?xml version="1.0" encoding="UTF-8"?>
<!DOCTYPE persona [
  <!ELEMENT persona ANY>
  <!ELEMENT nombre (#PCDATA)>
  <!ELEMENT ciudad (#PCDATA)>
]>

<persona>
  <nombre>Elsa</nombre> vive en Pamplona.
</persona>
```

Elementos vacíos en una DTD - **EMPTY**

Para declarar un elemento vacío en una DTD, hay que indicar que su contenido es **EMPTY**. Un ejemplo de ello podría ser el elemento "br" del HTML, el cual sirve para hacer un salto de línea y no tiene contenido. Por ejemplo, el siguiente documento XML sería válido:

```
<?xml version="1.0" encoding="UTF-8"?>
<!DOCTYPE etiquetas_html [
  <!ELEMENT etiquetas_html (br)>
  <!ELEMENT br EMPTY>
]>

<etiquetas_html>
  <br/>
</etiquetas_html>
```

Un elemento vacío puede tener atributos

EJEMPLO Aunque un elemento se declare vacío, no pudiendo contener texto ni otros elementos, sí puede tener atributos:

```
<?xml version="1.0" encoding="UTF-8"?>
<!DOCTYPE etiquetas_html [
  <!ELEMENT etiquetas_html (br)>
  <!ELEMENT br EMPTY>
  <!ATTLIST br descripcion CDATA #REQUIRED>
]>

<etiquetas_html>
  <br descripcion="Salto de línea"/>
</etiquetas_html>
```

En este ejemplo, para el elemento "br" se ha declarado el atributo descripcion de tipo CDATA (Character DATA), es decir, su valor puede ser una cadena de caracteres. Además, se ha indicado que el atributo es obligatorio escribirlo, #REQUIRED.

Elementos con cualquier contenido en una DTD - **ANY**

Cuando en una DTD se quiere declarar un elemento que pueda contener cualquier contenido (bien sea texto, otros elementos o una mezcla de ambos) esto se puede hacer indicando que su contenido es de tipo ANY.

EJEMPLOS: 1.- En el siguiente documento XML, el elemento "cualquier_contenido" contiene tres elementos "texto"

```
<?xml version="1.0" encoding="UTF-8"?>
<!DOCTYPE ejemplo [
  <!ELEMENT ejemplo (cualquier_contenido)>
  <!ELEMENT cualquier_contenido ANY>
  <!ELEMENT texto (#PCDATA)>
]>

<ejemplo>
  <cualquier_contenido>
    <texto>Texto1</texto>
    <texto>Texto2</texto>
    <texto>Texto3</texto>
  </cualquier_contenido>
</ejemplo>
```

2.- Fíjese que, definiendo la misma DTD, también sería válido el siguiente documento XML donde el elemento "cualquier_contenido" solo contiene texto:

```
<?xml version="1.0" encoding="UTF-8"?>
<!DOCTYPE ejemplo [
  <!ELEMENT ejemplo (cualquier_contenido)>
  <!ELEMENT cualquier_contenido ANY>
  <!ELEMENT texto (#PCDATA)>
]>

<ejemplo>
  <cualquier_contenido>Texto1. Texto2.Texto3</cualquier_contenido>
</ejemplo>
```

Elementos con cualquier contenido en una DTD - **ANY**

EJEMPLOS:

3.- Asimismo, el elemento "cualquier_contenido" podría contener una mezcla de texto y uno o más elementos.

```
<?xml version="1.0" encoding="UTF-8"?>
<!DOCTYPE ejemplo [
   <!ELEMENT ejemplo (cualquier_contenido)>
   <!ELEMENT cualquier_contenido ANY>
   <!ELEMENT texto (#PCDATA)>
]>

<ejemplo>
 <cualquier_contenido>Texto1<texto>Texto2</texto>Texto3</cualquier_contenido>
</ejemplo>
```

Elementos con cualquier contenido en una DTD - **ANY**

EJEMPLOS:

4.- Por otra parte, si el elemento "cualquier_contenido" estuviese vacío, el documento XML seguiría siendo válido:

```
<?xml version="1.0" encoding="UTF-8"?>
<!DOCTYPE ejemplo [
   <!ELEMENT ejemplo (cualquier_contenido)>
   <!ELEMENT cualquier_contenido ANY>
   <!ELEMENT texto (#PCDATA)>
]>

<ejemplo>
   <cualquier_contenido></cualquier_contenido>
</ejemplo>
```

En vez de <cualquier_contenido></cualquier_contenido>, también se puede escribir <cualquier_contenido/>.

Elementos con contenido de tipo texto en una DTD - **(#PCDATA)**

Para declarar en una DTD un elemento que pueda contener texto analizable, se tiene que indicar que su contenido es (#PCDATA), (Parsed Character Data):

EJEMPLOS: 1.- En el siguiente documento XML, el elemento "texto" contiene caracteres:

```
<?xml version="1.0" encoding="UTF-8"?>
<!DOCTYPE ejemplo [
  <!ELEMENT ejemplo (texto)>
  <!ELEMENT texto (#PCDATA)>
]>

<ejemplo>
  <texto>Este elemento solo contiene caracteres.</texto>
</ejemplo>
```

2.- Ahora bien, el elemento "texto" podría estar vacío y el documento XML seguiría siendo válido:

```
<?xml version="1.0" encoding="UTF-8"?>
<!DOCTYPE ejemplo [
  <!ELEMENT ejemplo (texto)>
  <!ELEMENT texto (#PCDATA)>
]>

<ejemplo>
  <texto></texto>
</ejemplo>
```

En vez de <texto></texto>, también se puede escribir <texto/>.

Secuencias de elementos en una DTD

En una DTD, un elemento (padre) puede ser declarado para contener a otro u otros elementos (hijos). En la sintaxis, los hijos (también llamados sucesores) tienen que escribirse entre paréntesis "()" y separados por comas ",".

EJEMPLO: Para declarar un elemento (padre) que contenga tres elementos (hijos), se puede escribir:

```
<!ELEMENT padre (hijo1, hijo2, hijo3)>
```

EJEMPLOS: 1.- En el siguiente documento XML, el elemento "persona" contiene a los elementos "nombre", "fecha_de_nacimiento" y "ciudad":

```
<?xml version="1.0" encoding="UTF-8"?>
<!DOCTYPE persona [
  <!ELEMENT persona (nombre, fecha_de_nacimiento, ciudad)>
  <!ELEMENT nombre (#PCDATA)>
  <!ELEMENT fecha_de_nacimiento (#PCDATA)>
  <!ELEMENT ciudad (#PCDATA)>
]>

<persona>
  <nombre>Iker</nombre>
  <fecha_de_nacimiento>26-12-1997</fecha_de_nacimiento>
  <ciudad>Valencia</ciudad>
</persona>
```

Secuencias de elementos en una DTD

EJEMPLOS: 1.- A su vez, los hijos también pueden tener sus propios hijos. Así, el elemento "fecha_de_nacimiento" puede contener, por ejemplo, a los elementos "dia", "mes" y "anio":

```
<?xml version="1.0" encoding="UTF-8"?>
<!DOCTYPE persona [
  <!ELEMENT persona (nombre, fecha_de_nacimiento, ciudad)>
  <!ELEMENT nombre (#PCDATA)>
  <!ELEMENT fecha_de_nacimiento (dia, mes, anio)>
  <!ELEMENT dia (#PCDATA)>
  <!ELEMENT mes (#PCDATA)>
  <!ELEMENT anio (#PCDATA)>
  <!ELEMENT ciudad (#PCDATA)>
]>

<persona>
  <nombre>Iker</nombre>
  <fecha_de_nacimiento>
    <dia>26</dia>
    <mes>12</mes>
    <anio>1997</anio>
  </fecha_de_nacimiento>
  <ciudad>Valencia</ciudad>
</persona>
```

Orden de los hijos de un elemento

En un documento XML, los elementos (hijos) de un elemento (padre), deben escribirse en el mismo orden en el que han sido declarados en la DTD

EJEMPLO: El siguiente documento XML no es válido, porque los elementos sucesores (hijos) del elemento "fecha_de_nacimiento" no se han escrito en el mismo orden que en la DTD.

```
<?xml version="1.0" encoding="UTF-8"?>
<!DOCTYPE persona [
  <!ELEMENT persona (nombre, fecha_de_nacimiento, ciudad)>
  <!ELEMENT nombre (#PCDATA)>
  <!ELEMENT fecha_de_nacimiento (dia, mes, anio)>
  <!ELEMENT dia (#PCDATA)>
  <!ELEMENT mes (#PCDATA)>
  <!ELEMENT anio (#PCDATA)>
  <!ELEMENT ciudad (#PCDATA)>
]>

<persona>
  <nombre>Iker</nombre>
  <fecha_de_nacimiento>
    <anio>1997</anio>
    <mes>12</mes>
    <dia>26</dia>
  </fecha_de_nacimiento>
  <ciudad>Valencia</ciudad>
</persona>
```

Cardinalidad de los elementos en una DTD

En una DTD, para definir el **número de veces que pueden aparecer los elementos** de un documento XML, se pueden utilizar los operadores de **cardinalidad** mostrados en la siguiente tabla:

Operadores de cardinalidad en DTD		
Operador	**Cardinalidad**	**Significado**
? (interrogación)	0-1	El elemento es opcional, pudiendo aparecer una sola vez o ninguna.
*** (asterisco)**	0-n	El elemento puede aparecer cero, una o más veces.
+ (signo más)	1-n	El elemento tiene que aparecer, obligatoriamente, una o más veces.

Nota: los elementos declarados en una DTD sobre los que no actúe ningún operador de cardinalidad, tendrán que aparecer obligatoriamente una única vez, en el o los documentos XML a los que se asocie.

Cardinalidad de los elementos en una DTD

Operador de cardinalidad "+" (signo más)

EJEMPLO: En el siguiente documento XML, el elemento "nombre" tiene que aparecer una o más veces. En este caso, aparece tres veces:

```
<?xml version="1.0" encoding="UTF-8"?>
<!DOCTYPE personas [
   <!ELEMENT personas (nombre+)>
   <!ELEMENT nombre (#PCDATA)>
]>

<personas>
   <nombre>Ana</nombre>
   <nombre>Iker</nombre>
   <nombre>Elsa</nombre>
</personas>
```

- Si sobre nombre no actuase el operador (+) el documento no sería válido, ya que, el elemento "personas" solo tendría que contener un elemento "nombre".
- En vez de (nombre+), también se puede escribir (nombre)+.

Cardinalidad de los elementos en una DTD

Operador de cardinalidad "*" (asterísco)

EJEMPLO: En la DTD interna del siguiente documento XML, se ha indicado que el elemento "nombre" tiene que aparecer una única vez. Ahora bien, el elemento "ingrediente" tiene cardinalidad (0-n), por tanto, puede aparecer cero, una o más veces:

```
<?xml version="1.0" encoding="UTF-8"?>
<!DOCTYPE receta_de_cocina [
  <!ELEMENT receta_de_cocina (nombre, ingrediente*)>
  <!ELEMENT nombre (#PCDATA)>
  <!ELEMENT ingrediente (#PCDATA)>
]>

<receta_de_cocina>
  <nombre>Tortilla de patatas</nombre>
  <ingrediente>Huevo</ingrediente>
  <ingrediente>Patata</ingrediente>
  <ingrediente>Aceite</ingrediente>
  <ingrediente>Sal</ingrediente>
</receta_de_cocina>
```

Cardinalidad de los elementos en una DTD

Operador de cardinalidad "?" (interrogación)

EJEMPLOS: 1.- En la DTD del siguiente documento XML, la cardinalidad del elemento "mayor_de_edad" es (0-1), siendo opcional su aparición, en este caso aparece

```
<?xml version="1.0" encoding="UTF-8"?>
<!DOCTYPE persona [
  <!ELEMENT persona (nombre, mayor_de_edad?)>
  <!ELEMENT nombre (#PCDATA)>
  <!ELEMENT mayor_de_edad EMPTY>
]>

<persona>
  <nombre>Iker</nombre>
  <mayor_de_edad/>
</persona>
```

2.- Así pues, el siguiente documento también es válido, en el que no aparece el elemento "mayor_de_edad".

```
<?xml version="1.0" encoding="UTF-8"?>
<!DOCTYPE persona [
  <!ELEMENT persona (nombre, mayor_de_edad?)>
  <!ELEMENT nombre (#PCDATA)>
  <!ELEMENT mayor_de_edad EMPTY>
]>

<persona>
  <nombre>Iker</nombre>
</persona>
```

Elementos opcionales en una DTD

En la DTD asociada a un documento XML, se pueden declarar elementos que contengan elementos opcionales. Para ello, se utiliza el operador de elección, representado por una barra vertical (|).

Operador de elección "|" (barra vertical)

EJEMPLO En el siguiente documento XML el elemento "articulo" **puede contener un elemento** "codigo" o un elemento "id"; obligatoriamente uno de ellos, **pero no ambos**:

```
<?xml version="1.0" encoding="UTF-8"?>
<!DOCTYPE articulo [
   <!ELEMENT articulo (codigo | id)>
   <!ELEMENT codigo (#PCDATA)>
   <!ELEMENT id (#PCDATA)>
]>

<articulo>
   <codigo>AF-33</codigo>
</articulo>
```

Elementos opcionales en una DTD

Operador de elección "|" y operador "*"

EJEMPLO: En la DTD del siguiente documento XML se indica que el elemento "articulos" puede contener varios elementos "codigo" e "id":

```
<?xml version="1.0" encoding="UTF-8"?>
<!DOCTYPE articulos [
  <!ELEMENT articulos (codigo | id)*>
  <!ELEMENT codigo (#PCDATA)>
  <!ELEMENT id (#PCDATA)>
]>

<articulos>
  <codigo>AF-32</codigo>
  <id>3891</id>
  <codigo>AF-50</codigo>
  <codigo>AF-89</codigo>
</articulos>
```

Obsérvese que, con el operador "*", en este ejemplo se ha indicado que el contenido del elemento "articulos" tiene cardinalidad (0-n). Por tanto, el elemento "articulos" puede:

- Estar vacío.
- Contener un elemento "codigo".
- Contener un elemento "id".
- Contener un elemento "codigo" y un elemento "id".
- Contener un elemento "codigo" y varios elementos "id".
- Contener un elemento "id" y varios elementos "codigo".
- Contener varios elementos "codigo" y varios elementos "id".

Nótese también que, dentro del elemento "articulos" pueden aparecer elementos "codigo" e "id" en cualquier orden.

Elementos opcionales en una DTD

Operador de elección "|" en una secuencia de elementos

EJEMPLO: En el siguiente documento XML, pueden aparecer cero o más elementos "articulo" que contengan un elemento "codigo" o un elemento "id", y obligatoriamente un elemento "nombre":

```
<?xml version="1.0" encoding="UTF-8"?>
<!DOCTYPE articulos [
  <!ELEMENT articulos (articulo)*>
  <!ELEMENT articulo ((codigo | id), nombre)>
  <!ELEMENT codigo (#PCDATA)>
  <!ELEMENT id (#PCDATA)>
  <!ELEMENT nombre (#PCDATA)>
]>

<articulos>
  <articulo>
    <codigo>AF-47</codigo>
    <nombre>Martillo</nombre>
  </articulo>
  <articulo>
    <id>2056</id>
    <nombre>Destornillador</nombre>
  </articulo>
</articulos>
```

Elementos opcionales en una DTD

Operador de elección "|" en una secuencia de elementos en una lista de opciones

EJEMPLO: En la DTD del siguiente documento XML se ha indicado que pueden aparecer cero o más elementos "localidad". En el caso de aparecer, cada uno de ellos contendrá los elementos "pais" y "ciudad", o alternativamente un elemento "codigo_postal":

```
<?xml version="1.0" encoding="UTF-8"?>
<!DOCTYPE localidades [
  <!ELEMENT localidades (localidad)*>
  <!ELEMENT localidad ((pais, ciudad) | codigo_postal)>
  <!ELEMENT pais (#PCDATA)>
  <!ELEMENT ciudad (#PCDATA)>
  <!ELEMENT codigo_postal (#PCDATA)>
]>

<localidades>
  <localidad>
    <pais>España</pais>
    <ciudad>Valencia</ciudad>
  </localidad>
  <localidad>
    <codigo_postal>31015</codigo_postal>
  </localidad>
</localidades>
```

Elementos opcionales en una DTD

#PCDATA en una lista de opciones permite contenido mixto

En DTD, se pueden declarar **elementos mixtos** que **permiten** la **combinación de contenido de texto y otros subelementos** dentro del mismo elemento **en cualquier orden**. Para declarar un elemento mixto, se utiliza la siguiente sintaxis, importante #PCDATA siempre va en primer lugar:

```
<!ELEMENT elemento (#PCDATA | subelemento)*>
```

En esta declaración, elemento es el nombre del elemento mixto. (#PCDATA | subelemento)* indica que **el elemento puede contener** contenido de **texto (#PCDATA) y/o el subelemento** especificado. El asterisco (*) al final indica que el contenido puede repetirse cero o más veces.

EJEMPLO: Fíjese que, el elemento "articulos" de este documento, puede contener contenido mixto, es decir, texto y otros elementos, en el orden que queramos

```
<?xml version="1.0" encoding="UTF-8"?>
<!DOCTYPE articulos [
  <!ELEMENT articulos (#PCDATA | codigo | id)*>
  <!ELEMENT codigo (#PCDATA)>
  <!ELEMENT id (#PCDATA)>
]>

<articulos>
  <id>8608</id>
  Teclado
  <codigo>AF-18</codigo>
  <codigo>AF-45</codigo>
  Disquetera
  <id>7552</id>
  <id>4602</id>
</articulos>
```

Elementos opcionales en una DTD

#PCDATA en una lista de opciones permite contenido mixto

EJEMPLO: Véase, en este último ejemplo, que el elemento "provincia" puede aparecer cero o más veces, pudiendo contener contenido mixto:

```
<?xml version="1.0" encoding="UTF-8"?>
<!DOCTYPE localidades [
  <!ELEMENT localidades (provincia*)>
  <!ELEMENT provincia (#PCDATA | ciudad | codigo_postal)*>
  <!ELEMENT ciudad (#PCDATA)>
  <!ELEMENT codigo_postal (#PCDATA)>
]>

<localidades>
  <provincia>
    Navarra
    <ciudad>Estella</ciudad>
    <codigo_postal>31015</codigo_postal>
    <ciudad>Tafalla</ciudad>
  </provincia>
  <provincia>
    Valencia
    <codigo_postal>46520</codigo_postal>
  </provincia>
</localidades>
```

1.2.2 Declaración de Atributos en una DTD

Mediante DTD se pueden declarar los tipos de atributos que pueden tener los elementos en un documento XML. La sintaxis para la declaración de tipos de atributos en DTD es la siguiente:

<!ATTLIST *elemento atributo tipo valorPredeterminado*>

Donde:

- *elemento* es el nombre del elemento al que se le va a declarar el atributo.
- *atributo* es el nombre del atributo que se está declarando.
- *tipo* especifica el tipo de datos del atributo. Puede ser CDATA, enumeración o ID/IDREF.
- *valorPredeterminado* indica el valor por defecto del atributo, si es que tiene uno. Puede ser #REQUIRED, #IMPLIED o #FIXED.

1.2.2 Declaración de Atributos en una DTD

Valor predeterminado: **#REQUIRED**, **#IMPLIED, #FIXED o Valor por defecto**

Además del nombre y el tipo del atributo también se puede especificar cómo debe comportarse el parser ante la presencia o ausencia de cierto atributo en un elemento del documento. Existen cuatro posibles alternativas:

- **#REQUIRED**: El ***atributo*** es de carácter ***obligatorio***.
- **#IMPLIED**: El ***atributo*** es ***opcional***.
- **#FIXED valor** entre comillas dobles (") o simples ('): El ***atributo*** toma siempre el ***valor fijo*** que hemos puesto entre comillas.
- **Valor por defecto**. Siempre va entre comillas dobles (") o simples ('). Si no se define ninguna de las alternativas anteriores el ***atributo*** será por defecto opcional (#implied), es decir ***si no ponemos ningún valor tomará el*** que hemos dado ***por defecto***, ***en caso contrario tomará el nuevo valor*** que le demos .

1.2.2 Declaración de Atributos en una DTD

Valor predeterminado: **#REQUIRED**, **#IMPLIED** o **#FIXED o Valor por defecto**

EJEMPLO:

El DTD está incluido en la parte superior, entre <!DOCTYPE ... [...]>. Define los atributos y las reglas que se aplican a los elementos.

Los elementos <articulo> tienen los siguientes atributos:

- **estado**: Atributo obligatorio **(#REQUIRED)** por lo tanto todos los articulos tiene que tener estado tomando uno de los tres valores
- **descripcion**: Atributo opcional **(#IMPLIED)**.
- **categoria**: Tiene un **valor por defecto** "electrónica". Si no se pone toma ese valor y si se pone otro como "educacion" toma el nuevo valor.
- **condicion**: Tiene un **valor fijo** "nuevo". Es decir todos los elementos <artículo> tendrán como valor del atributo condición siempre "nuevo
- **tipo**: Atributo enumerado con valores posibles físico, digital o servicio.
- **codigo**: Atributo de tipo texto (#PCDATA), que puede contener cualquier valor.

```
<?xml version="1.0" encoding="UTF-8"?>
<!DOCTYPE tienda [
  <!-- Atributo obligatorio: El atributo 'estado' debe estar presente -->
  <!ATTLIST articulo estado (disponible | agotado | oferta) #REQUIRED>
  <!-- Atributo opcional: 'descripcion' puede no aparecer -->
  <!ATTLIST articulo descripcion #IMPLIED>
  <!-- Atributo con Valor por defecto: Si no se define 'categoria', será "electrónica" -->
  <!ATTLIST articulo categoria "electrónica">
  <!-- Atributo con Valor fijo: 'condicion' siempre será "nuevo" -->
  <!ATTLIST articulo condicion #FIXED "nuevo">
  <!-- Atributo Enumerado: 'tipo' puede ser uno de los tres valores: 'físico', 'digital' o 'servicio' -->
  <!ATTLIST articulo tipo (físico | digital | servicio) #IMPLIED>
  <!-- Atributo de tipo texto (PCDATA): 'codigo' puede contener cualquier texto -->
  <!ATTLIST articulo codigo #PCDATA #IMPLIED>
]>

<tienda>
  <!-- Atributo #REQUIRED: 'estado' debe estar presente -->
  <articulo estado="disponible" descripcion="Televisor 4K" categoria="electrónica" condicion="nuevo" tipo="físico" codigo="A123">Televisor</articulo>
  <articulo estado="agotado" descripcion="Laptop Gamer" categoria="electrónica" tipo="digital" codigo="A456">Laptop</articulo>
  <articulo estado="oferta" descripcion="Smartphone 5G" categoria="electrónica" tipo="físico" codigo="A789">Smartphone</articulo>
  <!-- Atributo opcional #IMPLIED: No se especifica 'descripcion', pero no hay error por ser Opcional-->
  <articulo estado="agotado" tipo="físico" codigo="A404">Reloj Inteligente</articulo>
  <!-- Atributo con valor por defecto: 'categoria' es "electrónica" si no se especifica como en este caso -->
  <articulo estado="disponible" descripcion="Mouse Inalámbrico" tipo="físico" codigo="A101">Mouse</articulo>
  <!-- Atributo con valor fijo #FIXED: 'condicion' siempre es "nuevo", y en este caso lo hemos omitido -->
  <articulo estado="agotado" descripcion="Teclado Mecánico" categoria="electrónica" tipo="físico" codigo="A202">Teclado</articulo>
  <!-- Atributo Enumerado: 'tipo' puede ser "físico", "digital" o "servicio" -->
  <articulo estado="disponible" descripcion="Curso de Python" categoria="educación" condicion="nuevo" tipo="digital" codigo="A303">Curso</articulo>
  <!-- Atributo de tipo texto (PCDATA): 'codigo' puede contener cualquier texto -->
  <articulo estado="disponible" descripcion="Audífonos Bluetooth" categoria="electrónica" tipo="físico" codigo="xyz123">Audífonos</articulo>
</tienda>
```

1.2.2 Declaración de Atributos en una DTD

Tipo de datos del atributo: **CDATA**, Enumeración o ID/IDREF

En la especificación DTD se pueden declarar diferentes tipos de atributos para los elementos de un documento XML. Los tipos de atributos más comunes son CDATA, enumeración y ID/IDREF.

Atributo CDATA:

- CDATA permite cualquier tipo de datos de caracteres, incluyendo texto, números, símbolos y caracteres especiales.
- No se aplican restricciones especiales a los valores del atributo.

Sintaxis:

```
<!ATTLIST elemento atributo CDATA #IMPLIED>
```

EJEMPLO:

```
<!DOCTYPE galeria [
 <!ELEMENT galeria (imagen+)>
 <!ELEMENT imagen (#PCDATA)>
   <!ATTLIST imagen descripcion CDATA #IMPLIED>
   <!-- Define un atributo llamado descripcion para el elemento imagen.
   Este atributo es opcional (#IMPLIED) y puede contener cualquier string de texto.-->
]>
<galeria>
   <imagen descripcion="Una hermosa puesta de sol sobre el mar.">puesta_del_sol.jpg</imagen>
   <imagen descripcion="Un retrato en blanco y negro.">retrato_bn.jpg</imagen>
   <imagen>imagen_sin_descripcion.jpg</imagen>
</galeria>
```

1.2.2 Declaración de Atributos en una DTD

Tipo de datos del atributo: CDATA, **Enumeración** o ID/IDREF

Atributo Enumeración:

Sintaxis: <!ATTLIST elemento atributo **(valor1 | valor2 | valor3)** #IMPLIED>

- El atributo puede tener un conjunto específico de valores predefinidos.
- Solo se pueden usar los valores enumerados en el atributo.
- Se declaran los valores posibles dentro de paréntesis y separados por tuberías (|).
- Solo puede tomar uno de los valores
- No se pueden usar operadores de cardinalidad *, + o ? por que estos solo son válidos para la declaración de elementos

EJEMPLO:

```
<!DOCTYPE tienda [
  <!ELEMENT tienda (articulo+)>
  <!ELEMENT articulo (#PCDATA)>
    <!ATTLIST articulo estado (disponible | agotado | oferta) #IMPLIED>
    <!-- Define un atributo llamado estado para el elemento articulo, el cual
    puede tener uno de los tres valores: disponible, agotado, o oferta.
    Este atributo es opcional ya que hemos puesto #IMPLIED. -->
]>
<tienda>
    <articulo estado="disponible">Televisor 4K</articulo>
    <articulo estado="oferta">Laptop Gamer</articulo>
    <articulo estado="agotado">Smartphone XYZ</articulo>
    <articulo>Mouse Inalámbrico</articulo> <!-- Sin estado especificado -->
</tienda>
```

1.2.2 Declaración de Atributos en una DTD

Tipo de datos del atributo: CDATA, Enumeración o **ID/IDREF**

Atributo ID/IDREF:

Sintaxis:

```
<!ATTLIST elemento atributo ID #REQUIRED>
<!ATTLIST elemento2 atributo IDREF #IMPLIED>
```

Los atributos ID y IDREF se utilizan para establecer relaciones únicas entre elementos en un documento XML.

- El atributo **ID** debe ser un i**dentificador único en todo el documento**. Además, el valor de un atributo de tipo ID debe cumplir con las reglas de los NMTOKENs, que son cadenas de caracteres que solo pueden contener letras minúsculas, letras mayúsculas, números, puntos (.), guiones medios (-), guiones bajos (_), o el carácter dos puntos (:). Ten en cuenta también que un valor de ID debe comenzar con una letra.
- El atributo **IDREF** hace referencia al **valor de un atributo ID existente en otro elemento.**

1.2.2 Declaración de Atributos en una DTD

Tipo de datos del atributo: CDATA, Enumeración o **ID/IDREF**

Atributo ID/IDREF:

En este DTD:

1) <!ELEMENT red_social (persona+)>: Define que el documento contiene un elemento red_social que a su vez puede contener uno o más elementos persona.
2) <!ELEMENT persona (nombre)> y <!ELEMENT nombre (#PCDATA)>: Define que cada persona contiene un elemento nombre, el cual es un texto (PCDATA).
3) <!ATTLIST persona id ID #REQUIRED>: Define que cada persona debe tener un atributo id único, que es obligatorio.
4) <!ATTLIST persona amigo_id IDREF #IMPLIED>: Define que cada persona puede tener un atributo opcional amigo_id que hace referencia al id de otra persona en el documento.

El ejemplo de XML muestra cómo se estructura un documento que utiliza este DTD, donde cada persona tiene un ID único y puede referenciar el ID de otra persona como su amigo.

EJEMPLO:

```
<!DOCTYPE red_social [
  <!ELEMENT red_social (persona+)>
  <!ELEMENT persona (nombre)>
  <!ELEMENT nombre (#PCDATA)>
    <!ATTLIST persona id ID #REQUIRED>
    <!ATTLIST persona amigo_id IDREF #IMPLIED>
]>
<red_social>
   <persona id="p1">
      <nombre>Juan Pérez</nombre>
   </persona>
   <persona id="p2" amigo_id="p1">
      <nombre>Maria López</nombre>
   </persona>
   <persona id="p3" amigo_id="p2">
      <nombre>Carlos García</nombre>
   </persona>
</red_social>
```

1.2.2 Declaración de Atributos en una DTD

Declaración de varios atributos en un elemento

EJEMPLO: En la DTD del siguiente documento XML se ha indicado que el elemento "f1" puede tener tres atributos (**pais, fecha_de_nacimiento y equipo**):

- En este ejemplo, el atributo **equipo** es obligatorio escribirlo, #REQUIRED.
- El atributo **pais** es opcional y si no ponemos nada toma como valor por defecto España.
- Mientras que, el atributo **fecha_de_nacimiento** es opcional, #IMPLIED.

```
<?xml version="1.0" encoding="UTF-8"?>
<!DOCTYPE deportistas [
  <!ELEMENT deportistas (futbol | f1 | tenis)*>
  <!ELEMENT futbol (#PCDATA)>
  <!ELEMENT f1 (#PCDATA)>
    <!ATTLIST f1 pais CDATA "España">
    <!ATTLIST f1 fecha_de_nacimiento CDATA #IMPLIED>
    <!ATTLIST f1 equipo CDATA #REQUIRED>
  <!ELEMENT tenis (#PCDATA)>
]>

<deportistas>
  <f1 pais="Alemania" fecha_de_nacimiento="03/07/1987" equipo="Ferrari">Sebastian Vettel</f1>
  <f1 equipo="McLaren">Fernando Alonso</f1>
  <tenis>Rafael Nadal</tenis>
</deportistas>
```

En una DTD, cuando se declara más de un atributo para un elemento, como se ha hecho en este caso, no es necesario escribir varias veces <!ATTLIST, pudiéndose escribir, por ejemplo:

```
<!ATTLIST f1 pais CDATA "España"
          fecha_de_nacimiento CDATA #IMPLIED
          equipo CDATA #REQUIRED>
```

1.2.2 Declaración de Atributos en una DTD

Ejemplos de declaraciones de atributos

EJEMPLO: En la DTD del siguiente documento XML se ha indicado que el elemento "f1" puede tener el atributo "pais":

- Para el elemento "f1", **pais** es un atributo definido de tipo **CDATA** (Character DATA), es decir, su valor será una cadena de caracteres.
- Al no indicarse el país de Fernando Alonso, tendrá como valor por defecto es **"España"**.
- Para Sebastian Vettel, al atributo **pais** se le ha asignado **"Alemania"**, que es un valor distinto al **valor_del_atributo**, que por defecto es **"España"**.

```
<?xml version="1.0" encoding="UTF-8"?>
<!DOCTYPE deportistas [
  <!ELEMENT deportistas (futbol | f1 | tenis)*>
  <!ELEMENT futbol (#PCDATA)>
  <!ELEMENT f1 (#PCDATA)>
  <!ATTLIST f1 pais CDATA "España">
  <!ELEMENT tenis (#PCDATA)>
]>

<deportistas>
  <f1 pais="Alemania">Sebastian Vettel</f1>
  <f1>Fernando Alonso</f1>
  <tenis>Rafael Nadal</tenis>
</deportistas>
```

1.2.2 Declaración de Atributos en una DTD

Ejemplos de declaraciones de atributos

EJEMPLO: En la DTD del siguiente documento XML, se indica que el valor del atributo filmografia de un elemento "director", puede ser una lista de valores de atributos ID. En este caso, una lista de valores escritos en el atributo codpel de los elementos "pelicula" que aparezcan en el documento XML:

- Obsérvese que, los valores de la lista de valores de un atributo IDREFS, se escriben separados por un espacio en blanco.

```
<?xml version="1.0" encoding="UTF-8"?>
<!DOCTYPE cine [
  <!ELEMENT cine (peliculas, directores)>
  <!ELEMENT peliculas (pelicula)*>
    <!ELEMENT pelicula (#PCDATA)>
      <!ATTLIST pelicula codpel ID #REQUIRED>
  <!ELEMENT directores (director)*>
    <!ELEMENT director (#PCDATA)>
      <!ATTLIST director filmografia IDREFS #REQUIRED>
]>

<cine>
  <peliculas>
    <pelicula codpel="P1">Avatar</pelicula>
    <pelicula codpel="P2">Mystic River</pelicula>
    <pelicula codpel="P3">The Terminator</pelicula>
    <pelicula codpel="P4">Titanic</pelicula>
  </peliculas>
  <directores>
    <director filmografia="P2">Clint Eastwood</director>
    <director filmografia="P1 P3 P4">James Cameron</director>
  </directores>
</cine>
```

1.2.3 Declaración de Entidades en una DTD

Las **entidades** en DTD (Document Type Definition) son una forma de **almacenar y reutilizar información en** diferentes **partes del** documento **XML**.. Funcionan como "variables" que pueden contener texto, URLs o incluso partes de código XML.

En una DTD se pueden declarar entidades generales y paramétricas (de parámetro).
Las entidades generales pueden ser:

- Entidades generales internas analizables
- Entidades generales externas analizables
- Entidades generales externas no analizables

Las entidades paramétricas pueden ser:

- Entidades paramétricas internas analizables
- Entidades paramétricas externas analizables

Las entidades generales pueden utilizarse en el cuerpo de un documento XML y en su DTD. Sin embargo, las entidades paramétricas solo pueden utilizarse dentro de la DTD.

1.2.3 Declaración de Entidades en una DTD

Entidades generales internas analizables en DTD

Para declarar una entidad general interna analizable (parsed) en una DTD, se utiliza la siguiente sintaxis:

```
<!ENTITY    nombre_entidad "valor_entidad">
```

Donde:

- *nombre* es el nombre de la entidad
- *"valor"* es el texto o símbolo que representa la entidad.

1.2.3 Declaración de Entidades en una DTD

Entidades generales internas analizables en DTD

EJEMPLO: En la DTD del siguiente documento XML, se han declarado tres entidades (**escritor**, **obra** y **fecha**):
Obsérvese que, para referenciar a las entidades, se ha utilizado la sintaxis: &*nombre_entidad;*

```
<?xml version="1.0" encoding="UTF-8"?>
<!DOCTYPE textos [
  <!ELEMENT textos (texto)+>
  <!ELEMENT texto (#PCDATA)>

  <!ENTITY escritor "Miguel de Cervantes">
  <!ENTITY obra "El Quijote">
  <!ENTITY fecha "29/09/1947">
]>

<textos>
  <texto>&obra; fue escrito por &escritor;.</texto>
  <texto>&escritor; nació el &fecha;.</texto>
</textos>
```

Si este documento XML se visualizase en un navegador web, se vería algo parecido a:

Este fichero XML no parece tener ninguna información de estilo asociada.
Se muestra debajo el árbol del documento.

```
–<textos>
    <texto>El Quijote fue escrito por Miguel de Cervantes.</texto>
    <texto>Miguel de Cervantes nació el 29/09/1947.</texto>
  </textos>
```

1.2.3 Declaración de Entidades en una DTD

Entidades generales externas analizables en DTD

En una DTD se pueden declarar dos tipos de entidades generales externas analizables (parsed): privadas y públicas.
Para las **privadas** se utiliza **SYSTEM**, y para las **públicas** se utiliza **PUBLIC**.
La sintaxis en cada caso es:

```
<!ENTITY nombre_entidad  SYSTEM  "URI">
```

```
<!ENTITY nombre_entidad  PUBLIC  identificador_publico "URI">
```

URI (Uniform Resource Identifier) es la dirección del recurso externo, que puede ser una ruta local o una URL en la web.

1.2.3 Declaración de Entidades en una DTD

Entidades generales externas analizables privadas - SYSTEM

EJEMPLO: En la DTD del siguiente documento XML, se ha declarado la entidad **escritor**:

```
<?xml version="1.0" encoding="UTF-8"
standalone="no"?>
<!DOCTYPE textos [
  <!ELEMENT textos (texto)+>
  <!ELEMENT texto (#PCDATA)>

  <!ENTITY escritor SYSTEM "escritor.txt">
]>

<textos>
  <texto>El Quijote fue escrito por &escritor;.</texto>
</textos>
```

Suponiendo que el archivo "escritor.txt" contenga:

```
Miguel de Cervantes
```

En un navegador web (por ejemplo Edge) se podrá ver:

This XML file does not appear to have any style information associated with it. The document tree is shown below.

```
▼<textos>
   <texto>El Quijote fue escrito por Miguel de Cervantes.
   </texto>
   <texto>Miguel de Cervantes nació el 29/09/1947.</texto>
 </textos>
```

2. Como validar documento XML asociado a un DTD

Para validar un documento XML asociado a un DTD, puedes seguir estos pasos:

1)Crea un archivo XML válido que esté asociado a un DTD

2)Utiliza una herramienta o método para validar el documento XML con la DTD asociada:

- **Editor de código y algún complemento** que permita la validación de XML con DTD. Por ejemplo Visual Studio Code y la extensión XML de Red Hat.
- **Herramientas online** como truugo.com/xml_validator/.
- **Programa específico** como XMLSpy, Oxygen XML Editor o Altova XMLSpy.
- **Herramientas en línea de comandos**: Si estás trabajando en un entorno de línea de comandos, puedes utilizar herramientas como xmllint o xmlstarlet para validar el documento XML con la DTD asociada. Estas herramientas suelen ofrecer opciones específicas para realizar la validación y mostrar los resultados.